

Nama : Sudrajat
Kelas : TI-24-PA2
NPM : 242310013

Praktikum 7

1. Membuat Model dan Migration Tabel Users

Langkah pertama dalam membangun modul User Management adalah membuat representasi tabel di dalam database menggunakan Sequelize CLI. Perintah generator dijalankan untuk membuat kerangka model dan file migration.

Kode pada file migration (20260724122022-create-user.js) dan model (models/user.js) kemudian dimodifikasi agar atribut tipe data, batas karakter (varchar), serta aturan null-nya sesuai dengan spesifikasi struktur tabel yang diminta di modul.

Setelah penyesuaian selesai, perintah `npx sequelize-cli db:migrate` dieksekusi untuk mengimplementasikan struktur tersebut langsung ke dalam database MySQL.

```
D:\Kulyeah\Semester 4\Pemrograman Web\MERN-STACK\backend>npx sequelize-cli model:generate --name User --attributes email:string,username:string,password:string,is_active:boolean

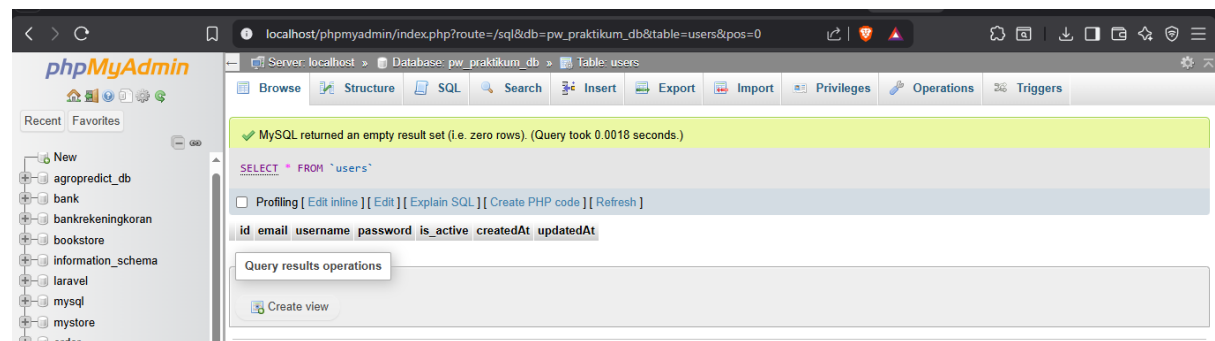
Sequelize CLI [Node: 26.3.0, CLI: 6.6.5, ORM: 6.37.8]

New model was created at D:\Kulyeah\Semester 4\Pemrograman Web\MERN-STACK\backend\models\user.js .
New migration was created at D:\Kulyeah\Semester 4\Pemrograman Web\MERN-STACK\backend\migrations\20260724122022-create-user.js .

D:\Kulyeah\Semester 4\Pemrograman Web\MERN-STACK\backend>npx sequelize-cli db:migrate

Sequelize CLI [Node: 26.3.0, CLI: 6.6.5, ORM: 6.37.8]

B injected env (7) from .env // tip: B custom filepath { path: '/custom/path/.env' }
Loaded configuration file "config\config.js".
Using environment "production".
== 20260724122022-create-user: migrating =====
== 20260724122022-create-user: migrated (0.110s)
```



2. Membuat Routing dan Controller API

Setelah tabel terbentuk, langkah selanjutnya adalah menangani logika bisnis (CRUD) melalui Controller dan mendefinisikan URL API melalui Routes.

Pada file controllers/userController.js, dibuat lima fungsi utama yaitu getAllUsers, getUserById, createUser, updateUser, dan deleteUser. Masing-masing fungsi dilengkapi dengan blok try-catch untuk error handling dan penentuan HTTP Status Code yang sesuai (misal: 200, 201, 404, 500).

Selanjutnya, fungsi-fungsi tersebut dipetakan ke dalam HTTP Method (GET, POST, PUT, DELETE) pada file routes/userRoutes.js. Routing ini kemudian didaftarkan ke file utama aplikasi (index.js) agar dapat diakses melalui endpoint /api/users.

usersController.js

```
const db = require("../models");
const User = db.User;

exports.getAllUsers = async (req, res) => {
  try {
    const users = await User.findAll();
    res.status(200).json({ success: true, count: users.length, data: users });
  } catch (error) {
    res
      .status(500)
      .json({
        success: false,
        message: "Gagal mengambil data user",
        error: error.message,
      });
  }
};

exports.getUserById = async (req, res) => {
  try {
    const user = await User.findByPk(req.params.id);
    if (!user) {
      return res
        .status(404)
        .json({ success: false, message: "User tidak ditemukan" });
    }
    res.status(200).json({ success: true, data: user });
  } catch (error) {
    res
      .status(500)
      .json({
        success: false,
        message: "Gagal mengambil data user",
        error: error.message,
      });
  }
};
```

```

    });
  }
};

exports.createUser = async (req, res) => {
  try {
    const { email, username, password, is_active } = req.body;
    const newUser = await User.create({ email, username, password, is_active });
  });
  res
    .status(201)
    .json({ success: true, message: "User berhasil dibuat", data: newUser });
} catch (error) {
  res
    .status(500)
    .json({
      success: false,
      message: "Gagal membuat user baru",
      error: error.message,
    });
}
};

exports.updateUser = async (req, res) => {
  try {
    const user = await User.findByPk(req.params.id);
    if (!user) {
      return res
        .status(404)
        .json({ success: false, message: "User tidak ditemukan" });
    }
    const { email, username, password, is_active } = req.body;
    await user.update({ email, username, password, is_active });
    res
      .status(200)
      .json({
        success: true,
        message: "Data user berhasil diperbarui",
        data: user,
      });
  } catch (error) {
    res
      .status(500)
      .json({
        success: false,
        message: "Gagal memperbarui data user",
        error: error.message,
      });
  }
};

```

```

    });
  }
};

exports.deleteUser = async (req, res) => {
  try {
    const user = await User.findByPk(req.params.id);
    if (!user) {
      return res
        .status(404)
        .json({ success: false, message: "User tidak ditemukan" });
    }
    await user.destroy();
    res.status(200).json({ success: true, message: "User berhasil dihapus" });
  } catch (error) {
    res
      .status(500)
      .json({
        success: false,
        message: "Gagal menghapus user",
        error: error.message,
      });
  }
};

```

usersRoutes.js

```

const express = require("express");
const router = express.Router();
const userController = require("../controllers/usersController");

router.get("/", userController.getAllUsers);

router.get("/:id", userController.getUserById);
router.post("/", userController.createUser);
router.put("/:id", userController.updateUser);
router.delete("/:id", userController.deleteUser);

module.exports = router;

```

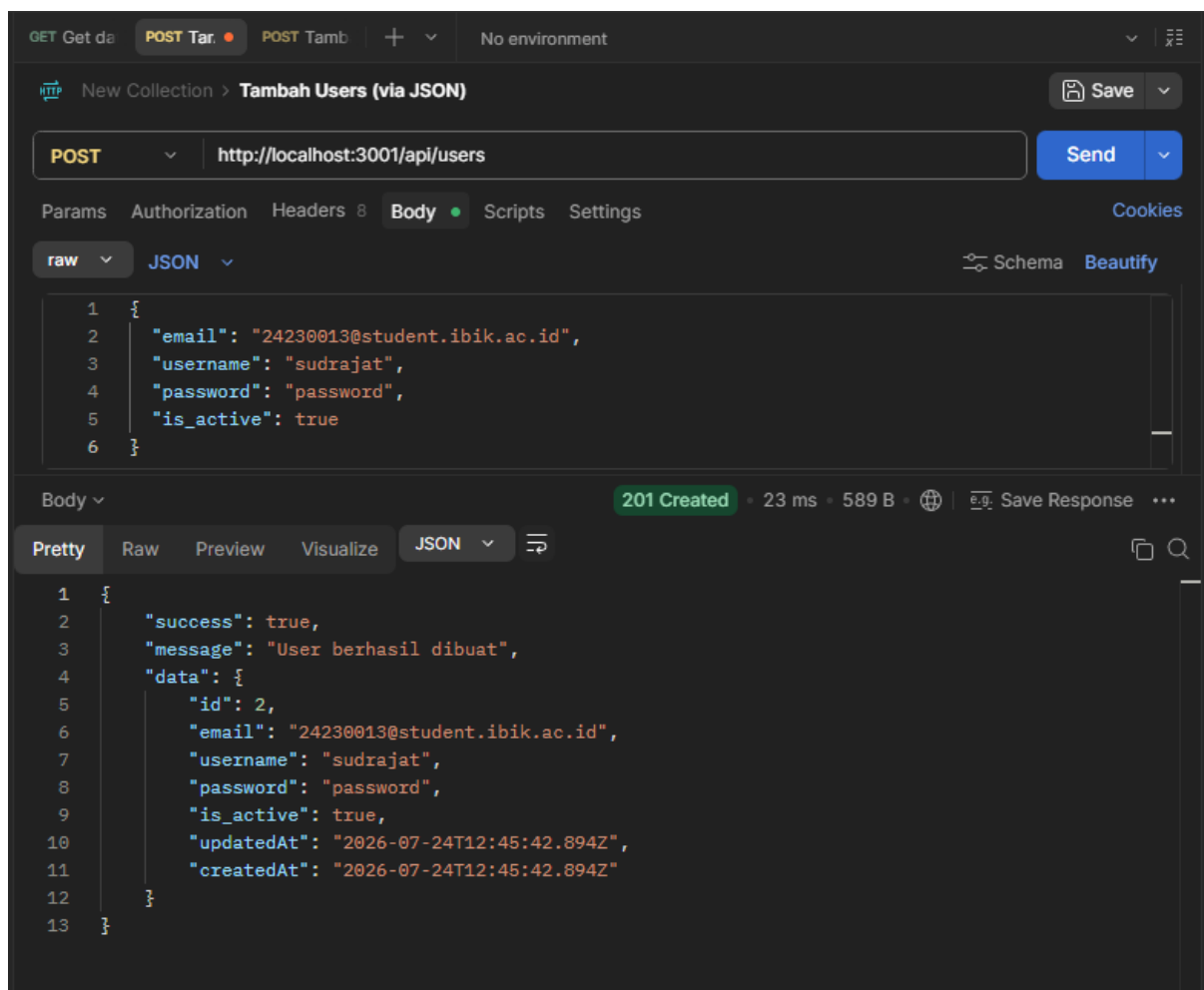
3. Pengujian API Menggunakan Aplikasi (Postman)

Tahap akhir adalah menguji fungsionalitas seluruh endpoint API yang telah dibuat menggunakan aplikasi Postman. Pengujian dilakukan untuk memvalidasi bahwa API dapat menerima request dan mengembalikan response JSON beserta HTTP Status Code yang akurat sesuai skenario (sukses atau gagal).

A. Pengujian Create User (POST)

Skenario: Mengirim data user baru melalui body request.

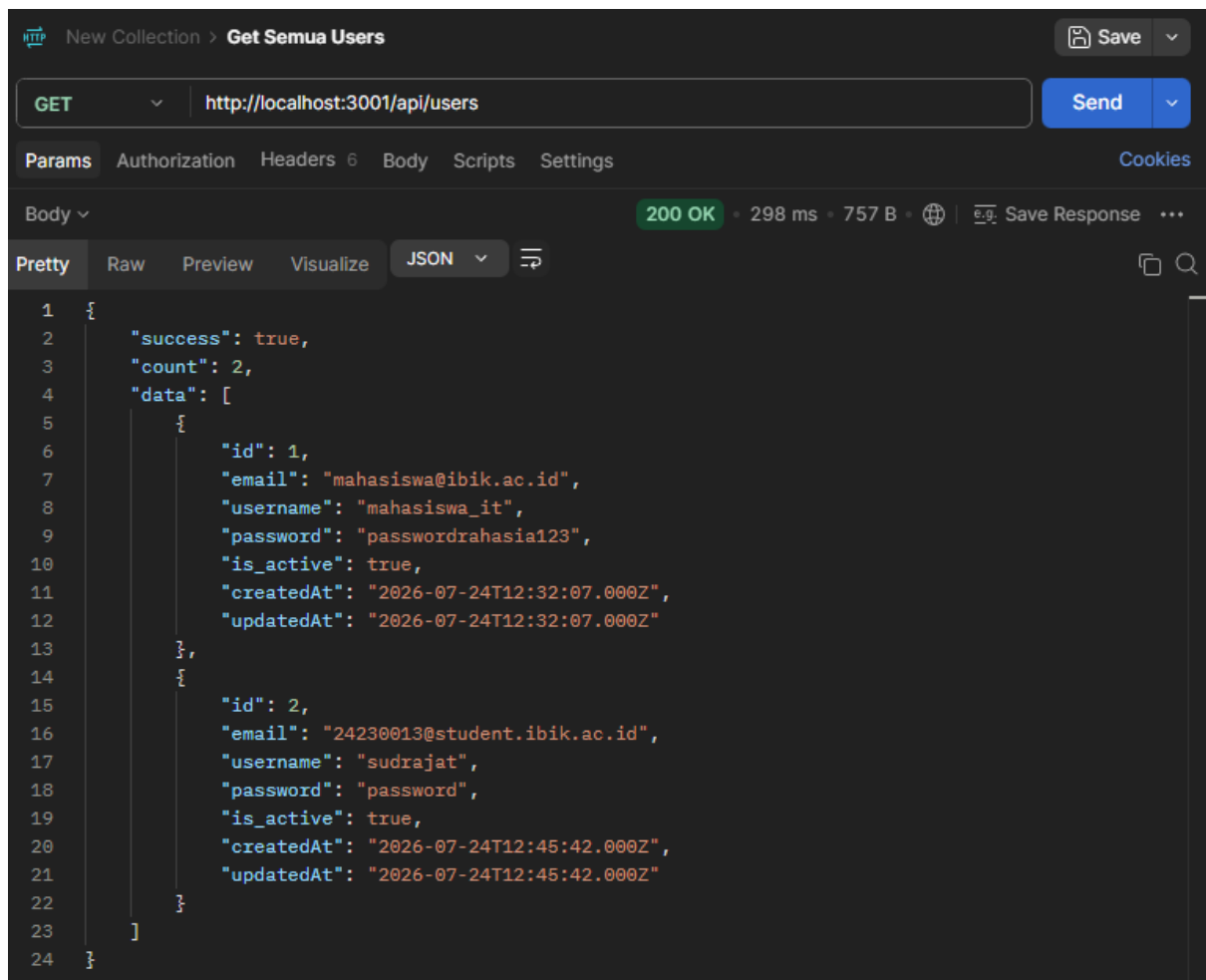
Ekspektasi: Sistem mengembalikan Status Code 201 Created dan data user tersimpan.



B. Pengujian Get All Users (GET)

Skenario: Mengambil daftar seluruh user.

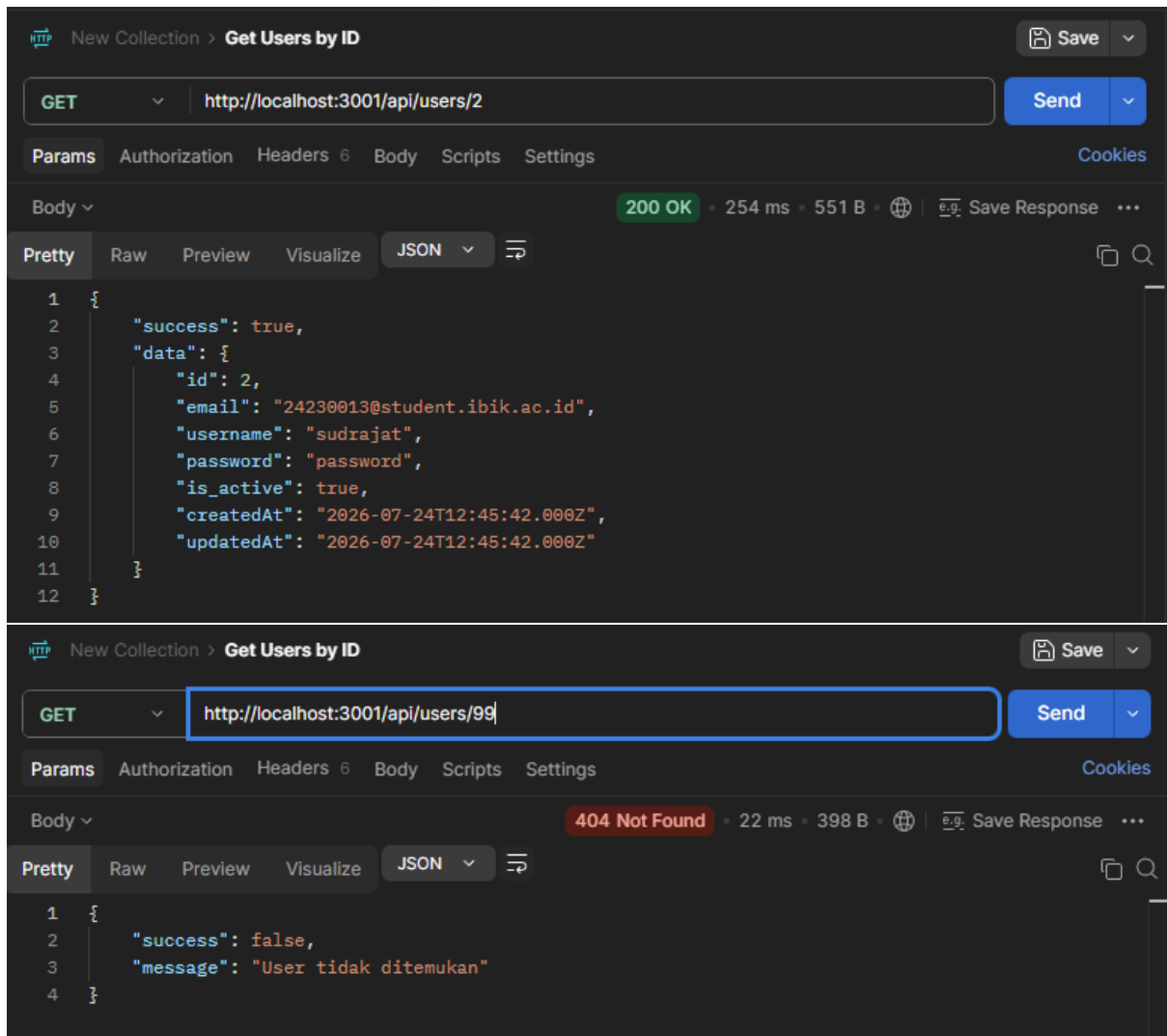
Ekspektasi: Sistem mengembalikan Status Code 200 OK beserta array data user.



C. Pengujian Get User by ID - Berhasil dan Gagal (GET)

Skenario: Memanggil detail user menggunakan ID yang valid dan ID yang tidak terdaftar.

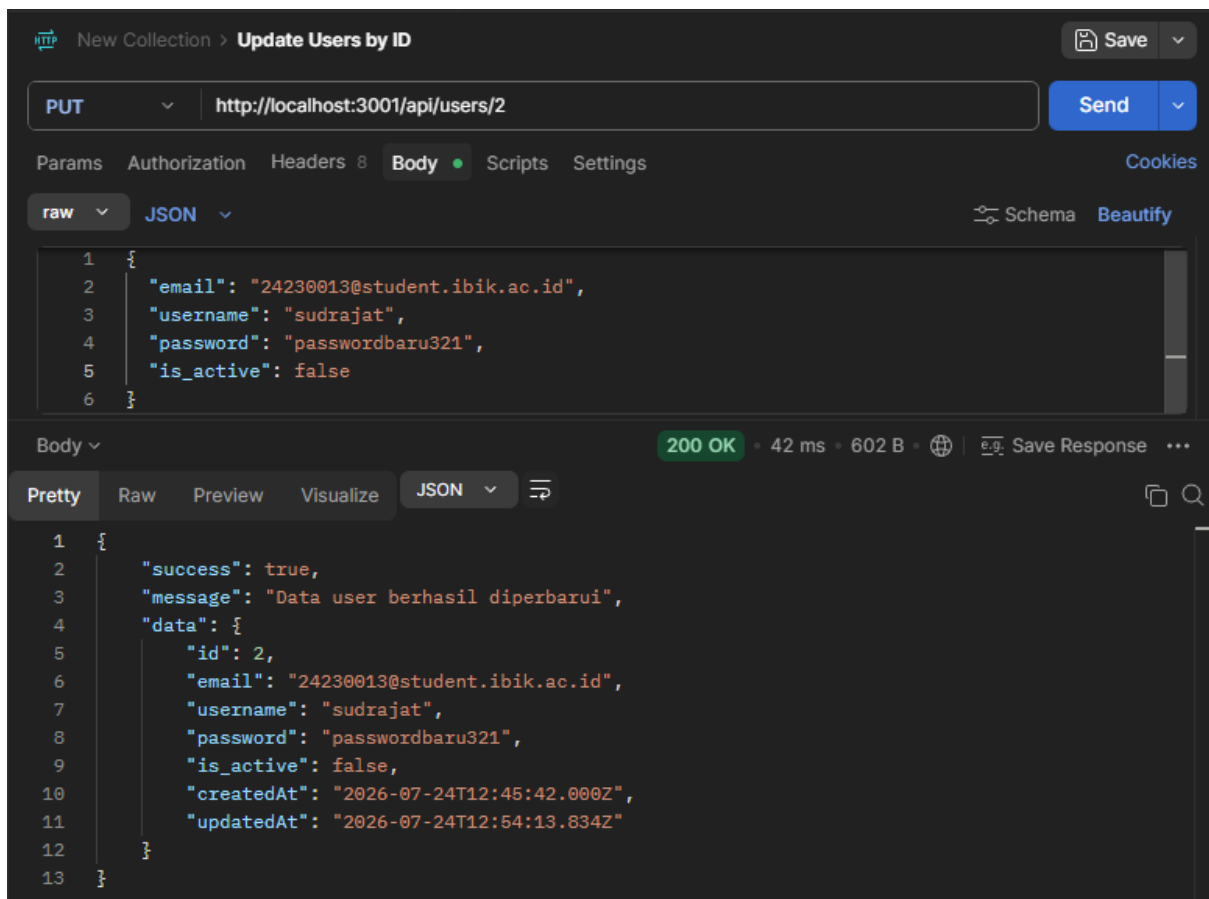
Ekspektasi: Status Code 200 OK untuk ID valid, dan Status Code 404 Not Found untuk ID yang tidak terdaftar.



D. Pengujian Update User (PUT)

Skenario: Mengirim data perubahan pada user tertentu.

Ekspektasi: Sistem mengembalikan Status Code 200 OK dan menampilkan data terbaru.



E. Pengujian Delete User (DELETE)

Skenario: Menghapus data user berdasarkan ID.

Ekspektasi: Sistem mengembalikan Status Code 200 OK dan pesan berhasil dihapus.

